

Spark 原理详解 - 第 17 讲

Spark on YARN 与资源模型：一条 spark-submit 命令背后到底发生了什么

从 Driver / ApplicationMaster / Executor / Container，到 Core / Memory / Task 并发

本讲目标 不背参数表，而是建立一个可以用于提交任务、看 YARN UI、看 Spark UI、估算资源和定位 OOM/资源不足问题的完整心智模型。

环境兼容说明 本课程仍按你当前的 Hadoop 3.3.5 + JDK 1.8 教学环境来理解 Spark 3.x。Spark 3.5.7 官方仍支持 Java 8/11/17；当前 Spark 4.x 已要求 Java 17，因此本课程不建议为了“追新版本”破坏现有虚拟机环境。

你需要最终回答的问题	本讲给出的答案
YARN 到底给 Spark 分配什么？	Container：一份 CPU/内存等资源配额
Executor 是谁启动的？	ApplicationMaster 向 YARN 申请，NodeManager 启动
Driver 在哪？	由 deploy-mode 决定：client 在提交端，cluster 在集群侧
executor-cores=2 有什么意义？	通常意味着该 Executor 同时大约能跑 2 个普通 Task
executor-memory=4g 为什么仍可能被 YARN 杀掉？	Container 总内存还包含 memory overhead 等额外部分

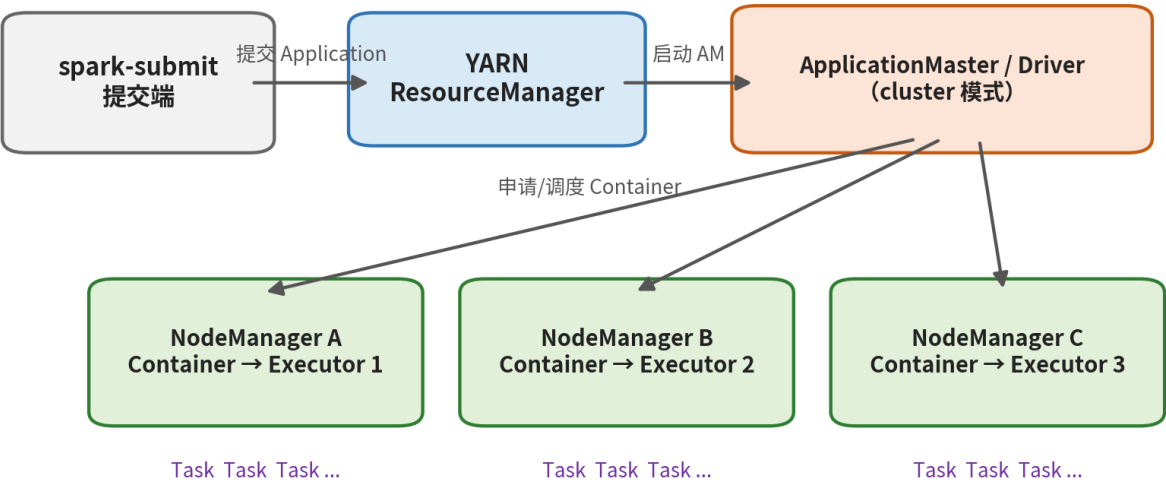
1. 为什么 Spark 需要 YARN ?

上一讲我们已经知道：Spark 自己负责把计算拆成 Job、Stage、Task。但 Spark 并不天然知道“集群上哪台机器还有 4 GB 内存、哪台机器还有 2 个 CPU 核”。这就是集群资源管理器要解决的问题。

一句话分工 Spark 负责“怎么算”；YARN 负责“给谁算、给多少资源、在哪台机器上启动进程”。

- YARN ResourceManager：整个集群的资源总调度者。
- NodeManager：每台工作节点上的“现场管理员”，负责启动和监控 Container。
- Spark Driver：计算总指挥，构建 DAG、切 Stage、调度 Task、跟踪执行状态。
- Spark Executor：真正干活的长期进程，运行 Task、缓存数据、参与 Shuffle。
- Container：YARN 批准的一份资源边界，不等于 Docker 容器，也不等于一台机器。

Spark on YARN：谁负责“算”，谁负责“分资源”？



核心关系：YARN 管资源；Spark Driver 管计算计划；Executor 真正执行 Task。

图 17-1 Spark on YARN 的总体角色分工

常见误区 “Executor = YARN Container” 不完全准确。更好的说法是：Executor 通常运行在 YARN 分配的 Container 中；Container 是资源配额/运行边界，Executor 是 Spark 进程。

2. deploy-mode：Driver 到底在里？

很多初学者第一次看到 --master yarn 和 --deploy-mode cluster 会把二者混为一谈。它们回答的是两个不同问题：

参数	它回答的问题	示例
----	--------	----

--master yarn	使用谁做集群资源管理？	YARN
--deploy-mode client/cluster	Driver 运行在哪里？	提交端 / 集群侧

Client 模式 vs Cluster 模式：Driver 到底在哪里？

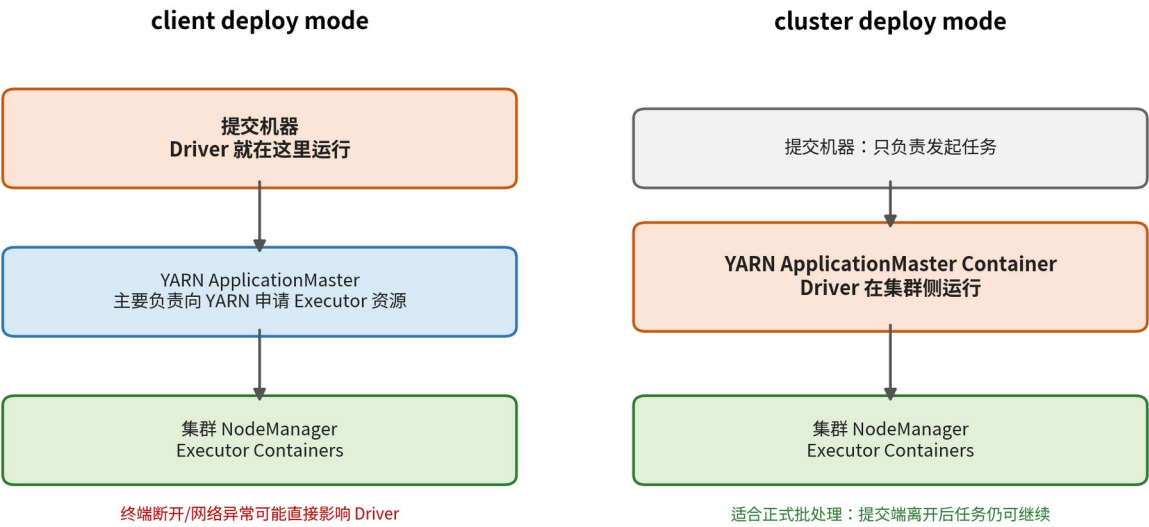


图 17-2 YARN client 与 cluster deploy mode

维度	client 模式	cluster 模式
Driver 位置	运行 spark-submit 的机器	YARN 集群侧 ApplicationMaster Container
提交端终端	通常需要保持可达	提交完成后可脱离
适合	交互调试、spark-shell、教学	正式批处理、服务器任务
Driver 日志	提交端直接看	YARN / Spark UI / yarn logs
风险	提交端掉线会影响 Driver	更适合长期批任务

最低闭环 看到一条 spark-submit 命令时，先找两件事：master 是谁？Driver 在哪？这一步能消除大量后续概念混乱。

3. 一条 spark-submit 命令的完整生命周期

```
spark-submit \
  --master yarn \
  --deploy-mode cluster \
  --num-executors 3 \
  --executor-cores 2 \
  --executor-memory 2g \
  --driver-memory 1g \
  resource_demo.py
```

现在不要把它当成 7 个参数去背，而是把它翻译成一句中文：

翻译 “把这个 Spark 应用交给 YARN；Driver 放在集群里；希望静态申请 3 个 Executor，每个 Executor 约 2 个计算核、2 GB JVM 堆内存；Driver 给 1 GB JVM 堆内存。”

spark-submit --master yarn --deploy-mode cluster：完整生命周期

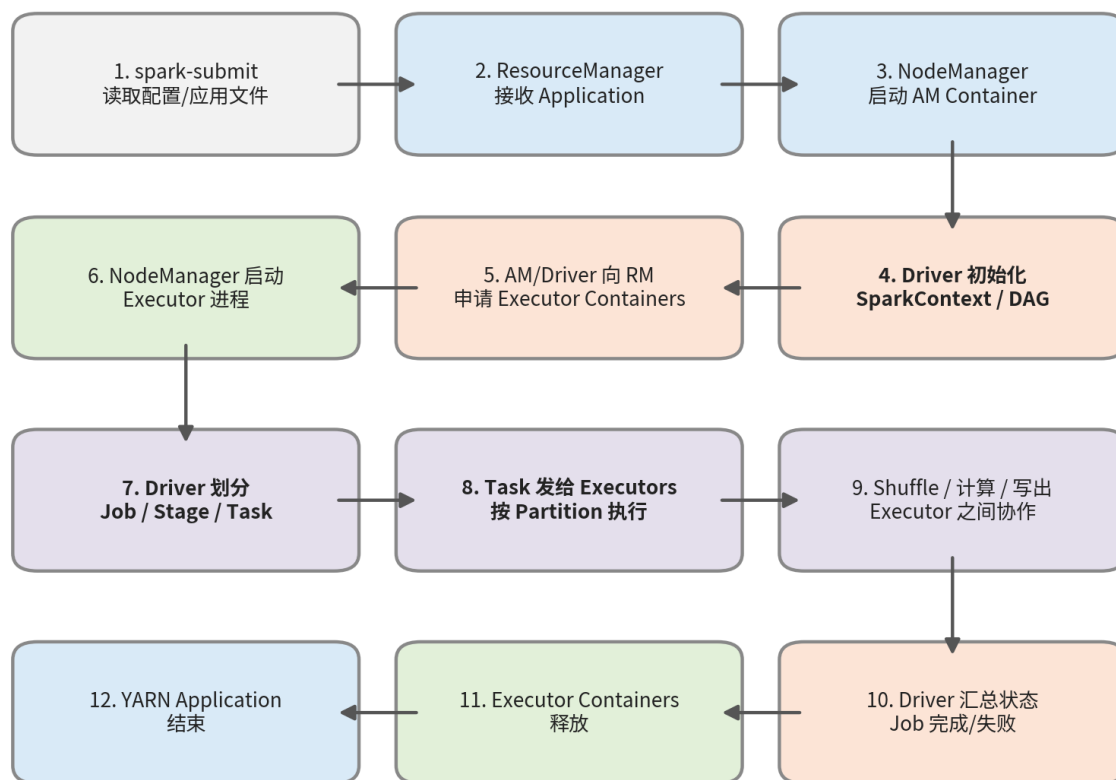


图 17-3 cluster 模式下从提交到结束的 12 步

1. spark-submit 读取应用、Spark 配置以及 Hadoop/YARN 配置。
2. 客户端向 ResourceManager 提交一个 YARN Application。
3. ResourceManager 选择某个 NodeManager，先启动 ApplicationMaster Container。
4. cluster 模式下 Driver 在集群侧初始化 SparkContext / SparkSession。
5. ApplicationMaster/Driver 根据配置向 YARN 请求 Executor Containers。
6. 各 NodeManager 在被分配到的节点上启动 Executor。

7. Driver 根据 Action 生成 Job，再根据依赖切 Stage 和 Task。
8. Driver 将 Task 分发给可用 Executor。
9. Executor 读取 Partition、计算、Shuffle、缓存或写结果。
10. Driver 持续接收 Task 状态，处理重试和后续 Stage。
11. 任务完成后 Executor Container 释放。
12. YARN 将 Application 标记为 FINISHED / FAILED / KILLED。

4. 最的源：Container → Executor → Core → Task

资源层级：Container → Executor → Core → Task

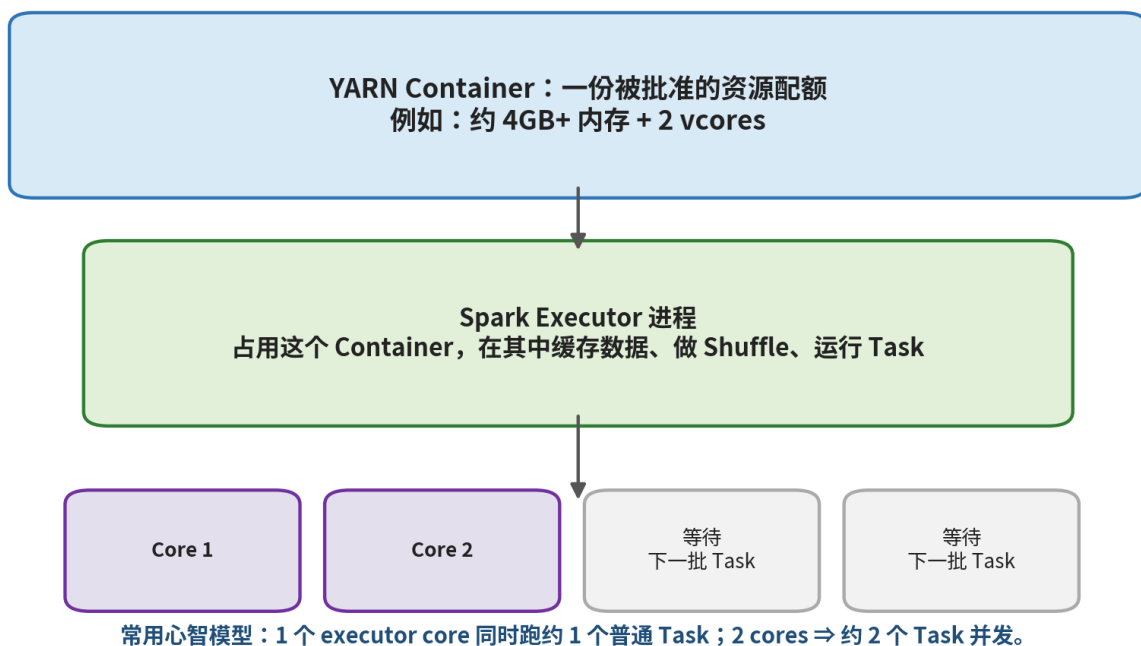


图 17-4 从 YARN Container 到 Spark Task 的资源层级

可以把它类比成一栋办公楼：YARN 先给 Spark 批一间办公室（Container），Spark 在里面安排一个长期工作的团队（Executor），团队有若干工位（cores），每个工位同一时刻通常处理一个任务单（Task）。

关键关系 一个 Executor 会连续执行很多 Task；Task 不是一个独立 JVM 进程，而是被调度到 Executor 里的执行单元。

概念	最通俗的理解	生命周期
YARN Container	批准的一份资源配额	可能伴随 AM 或 Executor
Executor	Spark 的工作进程	通常贯穿整个 Application
Core / vcore	Executor 可用的并行计算份额	资源配置
Task	对一个 Partition 的一次 Stage 级计算	短：做完就结束
Partition	数据的逻辑分片	决定 Task 数量的重要基础

5. executor-cores 到底控制什么？

教学阶段可以先采用一个非常实用的近似：普通 CPU Task 中，“1 个 executor core $\approx$ 同时运行 1 个 Task”。因此：

$3 \text{ executors} \times 2 \text{ cores/executor} \approx 6 \text{ 个 Task 同时运行}$

Partition 很多，但 Core 有限：Task 会分“波次”执行

例：一个 Stage 有 20 个 Partition $\rightarrow$ 20 个 Task；集群只有 6 个可用 executor cores

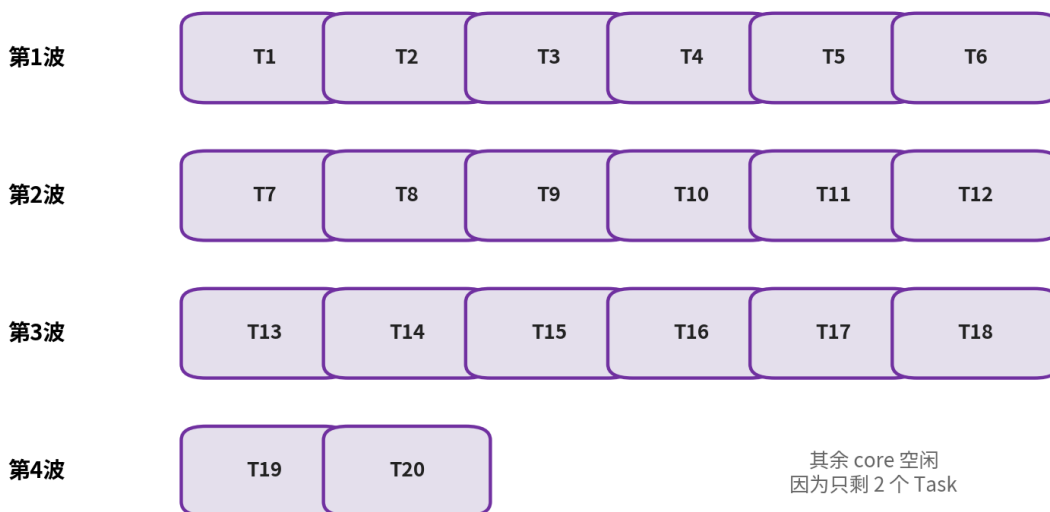


图 17-5 20 个 Task 在 6 个并发槽位上的波次执行

例子 如果某个 Stage 有 20 个 Partition，那么通常会生成约 20 个 Task。集群同时只能跑 6 个，就会形成 6 + 6 + 6 + 2 四波。

这也解释了一个常见现象：把 Partition 从 6 调到 600，并不会让只有 6 个 core 的集群瞬间获得 600 倍并行能力；它只会产生更多等待排队的小 Task。

不要机械追求并行度 Partition 太少会让 core 闲着；Partition 太多会增加调度、序列化、Shuffle 文件和小任务开销。最佳值取决于数据规模、Stage 类型和集群资源。

6. 一道最重要的资源计算题

假设你提交：

```
--num-executors 3
--executor-cores 2
--executor-memory 4g
```

先做最粗的第一层估算：

项目	估算
Executor 数量	3
每个 Executor 可并发 Task 数	约 2
Application 的 Task 并发上限（只看 Executor cores）	约 $3 \times 2 = 6$
JVM Executor Heap 总量	约 $3 \times 4\text{ GB} = 12\text{ GB}$
YARN 实际需要的总内存	大于 12 GB，因为还要算 memory overhead 等

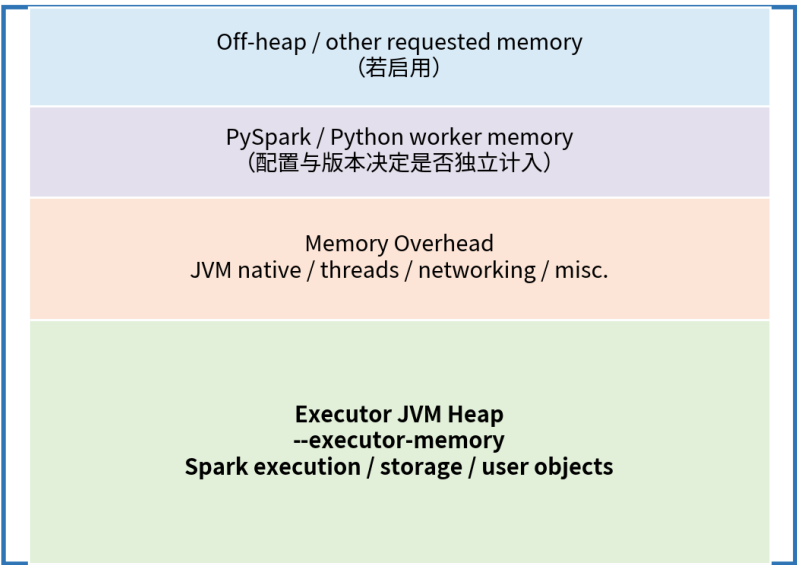
重要 “ $3 \times 4\text{ GB} = 12\text{ GB}$ ” 只算了 executor JVM heap，不等于 YARN 总资源请求。

而且这还不是说“集群有 12 GB 就一定能运行”。YARN 还要考虑：每个节点的可用资源、Container 最小/最大分配、Driver/AM、自身守护进程以及同时运行的其他 Application。

7. executor-memory 为什么不是 Container 的全部内存？

为什么 --executor-memory 不是 YARN Container 的全部内存？

YARN 关心的是 Container 总资源，不只 JVM -Xmx



因此“executor-memory=4g”不等于“只向 YARN 要 4g”。

图 17-6 Executor Container 中的主要内存组成（概念图）

`--executor-memory 4g` 主要指定 Executor JVM 的堆大小，但一个 Executor 进程运行时还会消耗其他内存，例如 JVM 自身的非堆/原生内存、线程栈、网络缓冲、序列化缓冲，以及 PySpark 的 Python worker 等。

为什么会出现“明明 4g 还被 YARN 杀掉”？ 因为 YARN 监控的是 Container 总资源。如果进程总使用量超过 Container 上限，可能出现 executor 被 kill、Container exceeded memory limits、exit code 137 等现象。

配置/概念	主要控制什么	常见误解
--executor-memory	Executor JVM heap	以为等于 Container 全部内存
memoryOverhead	非 JVM heap 等额外内存预算	以为可忽略
--driver-memory	Driver JVM heap	误以为会给 Executor
--executor-cores	每个 Executor 的 core 数	误以为等于机器物理核数

8. Driver 内存和 Executor 内存是两回事

Driver 不负责逐条处理所有分布式数据，但它掌握调度元数据、执行计划、Task 状态，并接收诸如 collect() / toPandas() 等拉回的数据。

- Executor OOM：通常说明某个分区太大、聚合状态过大、缓存太多、Shuffle/序列化压力大等。
- Driver OOM：常见于 collect()、toPandas()、收集过多结果、广播对象过大或调度元数据膨胀。
- 增加 executor-memory 不能解决 Driver OOM；增加 driver-memory 也不能解决某个 Executor 上的超大 Partition。

回忆第 12 讲 collect() 的危险就在这里：它把分布式结果集中回 Driver。真正需要保存海量结果时，通常写 HDFS/Parquet，而不是 collect 到本地。

9. 静态 Executor 与 Dynamic Allocation

到目前为止，我们用的是静态资源模型：提交时指定 executor 数量。Spark 也支持 Dynamic Allocation，让 Executor 数量随负载变化。

方式	特点	教学阶段建议
静态分配	Executor 数量相对固定，心智模型简单	先掌握，最适合本课程
Dynamic Allocation	可根据 backlog / 空闲情况增加或回收 Executor	理解概念即可，配置链更复杂

原理 动态分配并没有改变 Job → Stage → Task 的计算模型，它只是改变“同时有多少 Executor 可以接活”。

不要过早调参 在单机虚拟机或小型教学集群中，复杂的动态资源策略往往比它带来的收益更难理解。先把静态资源、Partition、Shuffle、数据倾斜弄清楚。

10. 在单节点 Hadoop 机里，些念成立？

成立。你的教学虚拟机即使只有一个 Ubuntu 节点，也可以运行 ResourceManager、NodeManager、HDFS 和 Spark on YARN。此时“分布式角色”仍然存在，只是很多进程最终落在同一台虚拟机上。

真实多机集群	单节点教学 VM
多个 NodeManager 分布在不同物理机	通常只有 1 个 NodeManager
Executor 可分布在多台机器	多个 Executor 可能都在同一 VM
网络 Shuffle 真正跨机器	可能变成本机进程间/本地网络栈传输
可研究真实性能扩展	主要用来学习机制与操作流程

实验定位 单节点 VM 的目标不是证明 Spark “比单机快”，而是让你观察 Application、Container、Executor、Stage、Task、日志和 UI 的真实生命周期。

11. 上机：正察 Spark on YARN

以下流程按 Hadoop 3.3.5 教学环境设计。每次上机仍遵循“启动前检查 → 启动服务 → 健康验证 → 再提交任务”。

```
# 1) 确认 Java / Hadoop / Spark
java -version
hadoop version
spark-submit --version

# 2) 启动 Hadoop 服务
start-dfs.sh
start-yarn.sh

# 3) 健康检查
jps
yarn node -list
hdfs dfsadmin -report | head -40
```

新建`resource\_demo.py`：

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('ResourceDemo').getOrCreate()
sc = spark.sparkContext

rdd = sc.parallelize(range(1, 1000001), 8)
result = (rdd
    .map(lambda x: (x % 10, 1))
    .reduceByKey(lambda a, b: a + b)
    .collect())

print(sorted(result))
spark.stop()
```

提交：

```
spark-submit \
  --master yarn \
  --deploy-mode cluster \
  --num-executors 2 \
  --executor-cores 1 \
  --executor-memory 1g \
  resource_demo.py
```

为什么这里参数很小？ 教学 VM 的资源有限。实验目标是观察机制，不是把虚拟机的 CPU 和内存全部吃满。

12. 任行察什？

```
# 查看正在运行的 YARN Application
yarn application -list

# 查看指定 Application 的状态
yarn application -status application_xxx_xxxx

# 如果开启了日志聚合，可查看完整容器日志
yarn logs -applicationId application_xxx_xxxx
```

观察位置	重点看什么	它回答的问题
YARN ResourceManager UI	Application、Container、Memory、vCores	YARN 给了多少资源？
Spark Web UI / History Server	Jobs、Stages、Tasks、Executors	Spark 怎样使用这些资源？
Executors 页面	Active Tasks、Storage、Shuffle、GC	哪个 Executor 压力大？
Stages 页面	Task 数、持续时间、Shuffle Read/Write	是否有长尾/倾斜/Shuffle？
yarn logs	Driver/Executor 异常堆栈	为什么失败？

两个 UI 不要混 YARN UI 更偏“资源与容器”；Spark UI 更偏“计算与数据流”。真正排障时经常需要两个一起看。

13. 常故障：象反推源

现象	优先怀疑	不要先做什么
Application 一直 ACCEPTED	YARN 没有足够资源/队列限制	不要先改业务代码
Executor lost / Container killed	内存超限、节点/进程异常	不要只盲目加 Partition
exit code 137 / exceeded memory	Container 总内存不够	不要只看 executor-memory
Driver OOM	collect/toPandas/过大结果或元数据	不要只加 executor-memory
只有少数 Task 很慢	数据倾斜、分区不均、热点 Key	不要只加 executor 数
大量 Task 很短但总体慢	Partition 过多/调度开销	不要继续无限 repartition

排查顺序 先判断是“资源拿不到”还是“拿到资源后算得慢”；再区分 Driver、Executor、Shuffle、Partition、数据倾斜。不要把所有慢任务都归因于“内存不够”。

14. 正需要掌握的 spark-submit 资源参数

参数	含义	先这样理解
--master yarn	使用 YARN 管理资源	“把 Spark 放到 Hadoop 集群里跑”
--deploy-mode client cluster	Driver 位置	教学交互常 client；正式批任务常 cluster
--num-executors N	静态 Executor 数量	有多少长期工作进程
--executor-cores N	每个 Executor 的 core 数	每个 Executor 同时能接多少普通 Task
--executor-memory 4g	每个 Executor JVM heap	不是 Container 总内存
--driver-memory 2g	Driver JVM heap	保护 Driver 的调度/收集空间
--queue name	YARN 队列	多用户集群里决定进入哪个资源队列

初学时不要追求所谓“最佳万能参数”。同一组参数换一个数据集、Partition 数、Join 策略、并发用户，效果都会变化。下一讲的性能优化，才会讨论如何用证据调参。

15. 最终心智模型：资源流与计算流要同时看

```

资源流：
YARN ResourceManager
  ↓ 分配 Container
NodeManager
  ↓ 启动
ApplicationMaster / Executor

计算流：
Driver
  ↓ Job
Stage
  ↓ Task
Executor cores
  ↓
Partition 数据
  
```

一句话总结 YARN 决定 Spark “能拿到多少机器资源”；Driver 决定 “这些资源接下来计算哪些 Task”；Executor 把 Task 真正跑完。

你现在应该能回答：

- 为什么 100 个 Partition 不等于 100 个 Task 同时运行？
- 为什么 executor-memory=4g 仍可能被 YARN 因内存超限杀掉？
- 为什么 client/cluster 模式的关键差异不是“速度”，而是 Driver 所在位置？
- 为什么 Executor OOM 和 Driver OOM 是两类完全不同的问题？
- 为什么 YARN UI 和 Spark UI 需要一起看？

16. 练习题（先独立思考，再看答案）

Q1：一个 Stage 有 48 个 Partition。你申请 4 个 Executor，每个 3 cores。粗略看最多同时运行多少个普通 Task？至少需要几波？

Q2: `--executor-memory 4g` 是否意味着 YARN 每个 Executor Container 只分配 4 GB? 为什么?

Q3: 为什么 `--master yarn --deploy-mode client` 中 Driver 不在 YARN Container 里做主要计算控制?

Q4: 任务一直停留在 YARN 的 ACCEPTED 状态, 还没有真正开始跑 Stage, 你首先应该排查 Spark SQL 逻辑还是资源申请?

Q5: 某 Stage 有 200 个 Task, 其中 199 个几十秒完成, 最后 1 个跑 15 分钟。增加 Driver memory 是否是第一选择?

参考答案

A1: $4 \times 3 \approx 12$ 个并发 Task; $48/12 = 4$ 波 (忽略调度差异)。

A2: 不是。4g 主要是 Executor JVM heap; Container 还要给 memory overhead 等额外内存。

A3: client 模式的定义就是 Driver 在提交端进程中运行, YARN AM 主要负责资源协商/Executor 管理。

A4: 先看 YARN 资源: 队列、可用 memory/vcores、Container 请求等。代码此时可能尚未真正进入计算。

A5: 不是。这个症状更像数据倾斜/热点分区/慢 Executor, 应先看 Stage Task 分布、Shuffle Read、Records、GC 等证据。

17. 与下一讲的连接：资源调优不是“越大越好”

理解资源模型以后, 下一个自然问题就是: 到底怎样配置 Executor 数、core 数、Partition 数、内存、缓存和 Join 策略, 才能让任务真正更快?

下一讲 第 18 讲将进入 Spark 性能优化: Partition 与并行度、Shuffle、数据倾斜、Broadcast Join、cache/persist、序列化、GC、小文件和 Spark UI 证据驱动调优。

参考资料 (官方)

- [Apache Spark 3.5.7 Documentation - Running Spark on YARN](#)
- [Apache Spark 3.5.7 Documentation - Cluster Mode Overview](#)
- [Apache Spark 3.5.7 Documentation - Configuration](#)
- [Apache Hadoop YARN Documentation - ResourceManager / NodeManager / Container](#)